---
name: three-column-thinking
description: "Decompose any chat input — conversational or deterministic — int
o atomic claims and answer as a synchronised three-column payload: (1) plain E
nglish reasoning, (2) exact mathematics over $\mathbb{Z}/\mathbb{Q}/\mathbb{F}_2$, (3) a UBP-compliant execu
table Python script (no floats, no random, no SHA-256 in computation, stdlib onl
y) that re-derives column 2 in an isolated interpreter. Includes a post-verification
optimization loop (step reduction, re-verify each iteration) and the cent_compiler
(£→¢→Δ) cross-domain isomorphism mode: compile a problem into the 22-primi
tive ¢ vocabulary and solve it via analogies across electrical/mechanical/fluidic/
thermal/optical/digital/spintronic/biochemical domains. Platform-dependent Lea
n tier, probed at run time (`lean_probe.py` checks tactic and notation availability
before lemmas are written): each formalizable step becomes a lemma in a lake-
build-ready project, proved where possible, `sorry` with a named gap where not.
Methodology from the Geometric Language Machine (GLM), E. R. A. Craig (Digit
alEuan), New Zealand — github.com/DigitalEuan/GLM."

Three Column Thinking (TCT) — UBP discipline + Lean verification tier

Methodology: Geometric Language Machine (GLM), E. R. A. Craig (DigitalEuan),
Auckland, New Zealand — https://github.com/DigitalEuan/GLM (§2.2–§2.3, §17,
§19.3, §24). Every answer is a synchronised three-column payload: the three
columns describe the **identical logical steps**, not three different answers.
A result is `VERIFIED` only when an independent re-derivation agrees. The Lean

tier formalises each step where possible and records the rest as `sorry` with a
named gap — that sorry set is exactly the development required for the model t
o
be fully described.

Table of Contents
1. [When to trigger](#1-when-to-trigger)
2. [The UBP discipline (mandatory in all computation)](#2-the-ubp-discipline-ma
ndatory-in-all-computation)
3. [Decomposition pipeline — any input, conversational or deterministic](#3-dec

omposition-pipeline--any-input-conversational-or-deterministic)
4. [The three-column contract](#4-the-three-column-contract)
5. [Verification tiers](#5-verification-tiers)
6. [Output format](#6-output-format)
7. [Named refusal taxonomy](#7-named-refusal-taxonomy)
8. [Lean lemma patterns](#8-lean-lemma-patterns)
9. [Worked examples](#9-worked-examples)
10. [Bundled scripts](#10-bundled-scripts)
11. [The optimization loop (post-VERIFIED iteration)](#11-the-optimization-loop-post-verified-iteration)
12. [cent_compiler cross-domain mode (£ → ¢ → Δ)](#12-cent_compiler-cross-domain-mode--)
13. [Bundled scripts — self-contained source (materialize if absent)](#13-bundled-scripts--self-contained-source-materialize-if-absent)
14. [Optimal construction — the four conditions (runbook)](#14-optimal-construction--the-four-conditions-runbook)

1. When to trigger

Any question the user wants resolved into provable vs. open parts: calculate, derive, solve, prove, unit conversion, count, compare, bounded estimate, checkable numeric claims, and conversational questions containing a deterministic sub-claim. Purely opinion-only questions with no decomposable claim are out of scope; everything else is decomposed.

2. The UBP discipline (mandatory in all computation)

The Universal Binary Principle (GLM §2.2) enforces these as standing rules with

instruments, not preferences. Apply them to Column 3 and to every intermediate

computation:

Commitment	Operational ban
No floats — `fractions.Fraction` (or `int`) only	No SHA-256 hashes in computati

on |
Exact arithmetic only — no rounding, no truncation	No XOR except over declared $\mathbb{F}_2$ sites
Standard library only	No random seeds / no randomness
Re-derived, falsifiable facts only	No disruptive (non-substrate) ops except at declared sites

Correspondence to GLM standing rules (§24): no floats = directive D7 (checked by
parsing every module); XOR and non-substrate operations allowed only at declared
sites (D9, D11); SHA-256 confined to integrity only, never meaning (D3) — if you need a hash for integrity, isolate it in a separate module and say so.

Why. The iteration-drift study shows under $X_{n+1} = (p+1)/p \cdot X_n - 1/p$, IEEE-754 float64 loses all semantic information by step 200; the drift is deterministic, a hardware consequence, not a model failure. Rounding and truncation are the same class of loss. Random and disruptive operations break reproducibility and therefore re-derivability. If a computation genuinely cannot be exact, state the error bound explicitly and treat the result as measured, not exact — and prefer refusal with a named reason over a lossy answer.

3. Decomposition pipeline — any input, conversational or deterministic

1. **Break the input into atomic claims.** Each is one step: a given, a definition, a derivation, or an empirical premise.
2. **Classify each claim.**
   - *Formalizable, closed-form* (concrete arithmetic, finite decidable facts) → Lean theorem with a pure-core proof: `decide` on closed `Nat`/`Int`/`Bool` facts (cross-multiply rationals into `Nat` first, §8) or `rfl`; `omega` and `simp` are also pure core. Probe the toolchain before writing (§8).
   - *Formalizable but unproven* (symbolic algebra needing mathlib, unproven identity) → Lean theorem with `sorry` + a `-- GAP:` line naming the required development.
   - *Empirical / conversational premise* (physical law, user assertion,

real-world fact) → `axiom` with a note; never a sorry theorem.
   - *Under-specified / missing data* → named refusal (§7); that part is not formalised.
3. **Write the three columns** (§4) over the steps.
4. **Scaffold the Lake project** with `scripts/scaffold_lean_project.py` from a JSON spec of the classified claims. Pure core Lean by default (no network, builds anywhere); `--with-mathlib` for symbolic proofs.
5. **Verify.** Column 3 in isolation (`scripts/verify_column3.py`). If `lake` is on PATH, run it via `scripts/lean_ledger.py --build`; otherwise report "not run; project ready for `lake build`".
6. **Report the verdict.** Python: VERIFIED/MISMATCH. Lean: N proved, M sorry, A axioms, with sorry gaps listed. Each sorry = development required.

For chained questions: every step carries its own three columns and its own Le an
lemma; the chain's Column 3 re-checks every step. Steps the user did not ask fo r
but the chain needs are marked **stitched**.

4. The three-column contract

Column 1 — Plain English. One sentence per atomic claim, same order as Column 2, written to be readable back from the mathematics (declarative, prefix-first phrasing: "the sum of A and B", "the negation of A", "the fraction P over Q"). No freeform prose unmappable to a math step.

Column 2 — Mathematics. Same steps as exact equations over $\mathbb{Z}$ / $\mathbb{Q}$ / $\mathbb{F}_2$. State givens, variables, domains, each derivation step, final result. Float only with a declared error bound (see §2).

Column 3 — Executable Python (UBP-compliant). Self-contained, stdlib-only

script re-deriving Column 2: `fractions.Fraction` / `int`, no floats, no rounding, no random, no SHA-256 in computation, no XOR outside $\mathbb{F}_2$. Include `assert` cross-checks between independent derivation paths. Print `KEY=VALUE

`.
It is the independent witness — no shared state with your reasoning.

5. Verification tiers

- **Tier 1 — Python (always).** `scripts/verify_column3.py` runs Column 3 in a fresh subprocess. Exit 0 = VERIFIED, 1 = MISMATCH, 2 = error/timeout.
- **Tier 2 — Lean (when toolchain present, or on user request).** Always emit the Lake project as the usable artifact even if `lake` is not installed here. `sorry` compiles (warning only), so `lake build` always succeeds; **0 sorry = fully described**. Run the build whenever a toolchain is present — an uncompiled `decide` proof is provisional (Run D's kernel refuted one translation whose Python-side value was correct), and `lean_probe.py` (§13) checks the tactic landscape in ~30 s before you write lemmas. `scripts/lean_ledger.py` scans and reports the sorry/axiom count and, with `--build`, runs `lake build`. The scaffold pins no version by default — it writes `lean-toolchain` = `leanprover/lean4:stable`, which `elan` resolves automatically, so no toolchain guess is needed; pin a version with `--toolchain` only for reproducibility.

6. Output format

```
**Givens.** ... (if the question is "why/how" and supplies no givens, choose and explicitly declare *illustrative model values* — label them as such; the conceptual/mechanism answer must not depend on them)
**Reading.** the licensed interpretation(s) of ambiguous wording, declared *before* decomposition (e.g. "after Wednesday" = the immediate successor day, not any later day; "become large" = expand on heating below 4 °C). One licensed reading → proceed with it named. Two conflicting licensed readings with no tie-break → `READINGS_DISAGREE` (§7). Readings are licensed like axioms: named, never silent.
**Domain.** $\mathbb{Z}$ / $\mathbb{Q}$ / $\mathbb{F}_2$ / float with declared bound ...
**UBP compliance.** Column 3: Fraction/int only, no floats/random/SHA — or note any declared exception site.
**Decomposition.** N atomic claims: K proved, M sorry, A axioms.
```

Step	Plain English	Mathematics	Python (this step)	Lean
1	...	...	...	theorem ... (proved) / sorry + gap / axiom

Python column: a one-line witness (`assert P == V*V/R`) when it fits; when the witness is multi-line, put the cross-check *name* or the KEY it sets in the cell and the full witness in the Column-3 script below — never truncate mathematics to fit the column.

Coverage map. Lean lemmas cover k of N decomposed claims; Python covers
all N. Steps without a Lean lemma are listed by number with a reason (e.g. "5: interval arithmetic — no pure-core irrational lemma"; "7: reporting only"). A coverage hole is not an error; an unlisted one is.

Full Column 3 script:
```python
...
```

Lean project: `<path>/` (lakefile.lean, lean-toolchain, TCT/Basic.lean, SORRY_LEDGER.md)
Verification: Python: VERIFIED ... / MISMATCH ...
Lean: N proved, M sorry, A axiom. sorry gaps: ... (development required). lake build: succeeded / not run (ready for `lake build`).
```

## 7. Named refusal taxonomy

Refuse with a named reason only when even decomposition cannot close a part.
State the specific fact that triggers it.

| Reason | When |
|---|---|
```

`UNKNOWN_ROW`	Object/entity asked about is not in the data/source.
`MISSING_FIELD`	Object exists but a needed field is missing.
`DIFFERENT_SCALES`	Quantities on different scales/units, no declared conve rsion.
`COLUMN_WITH_HOLE`	A needed value at a step is absent; chain cannot clo se.
`READINGS_DISAGREE`	Two licensed readings of same quantity disagree, no tie-break rule.
`INCONSISTENT_GIVENS`	Givens contradict (e.g. V=12, I=5, R=4 simultaneousl y).
`DERIVATIONS_DISAGREE`	Two derivation paths from same givens to same ta rget differ.
`UNDERDETERMINED`	Givens do not determine a unique answer (state free v ariables).
`INTEGER_UNDECIDED`	Integer-domain decision past a declared, bounded se arch.
`EXACT_ARITHMETIC_UNAVAILABLE`	Cannot compute exactly; float error un bounded/unacceptable.
`WOULD_BE_GUESS`	No exact derivation path; any answer would be a guess. Catch-all.
`UNKNOWN_QUERY_KIND`	Question matches no computation pattern; TCT n ot forced.

Honest refusal beats a confidently wrong answer. A `MISMATCH` caught by Colu mn 3
is a win for the method — fix the wrong column and re-verify.

8. Lean lemma patterns

Probe first, then write. Environment claims rot with the toolchain — notation, tactics and constants appear and disappear between versions. Before writing lemmas, run `scripts/lean_probe.py` (§13; ~30 s, needs only `lean`) and **believe the probe over this table**. The landscape below is the verified pure-core baseline (Lean 4.35.0 / Lake 5.0.0, no imports, no network):

Pattern	Core 4.35	Notes / use
`decide` on closed `Nat`/`Int`/`Bool`	✔	the workhorse
`rfl`	✔	definitional equalities
`omega`	✔	Int linear arithmetic incl. `%` by constants; goal-shape sensitive — proves Int-mod successor laws, fails Nat-mod goals with negated conclusions
`simp`	✔	simplification
`ℚ` / `ℕ` notation	✘	not in core — elaborates as fresh type variables; write `Rat` / `Nat`
`decide` / `rfl` on `Rat`	✘	kernel reduction stuck (`instDecidableEqRat`) — cross-multiply (below)
`native_decide` on `Rat`	✔	works, but compiler-trusted, not kernel-checked — label it
`norm_num`, `ring`, `linarith`, `field_simp`	✘	Mathlib only — `--with-mathlib`, or sorry + gap
`Nat.factorial`	✘	not in core — inline values or define your own
`Real.pi`	✘	not in core — geometry axiomatises the constant and says so
`Nat` bitwise XOR	△	`decide` succeeds on closed literals; fails on goals with variables

The kernel-checked rational route: cross-multiplication. To prove an equation between exact rationals `a/b = c/d`, clear denominators and state the `Nat`/`Int` fact; name the lemma after the reading it witnesses, so the translation is self-documenting:

```lean
theorem power_cross : 12 * 12 = 36 * 4 := by decide  -- P = V²/R = 36 W, × b·d
```

Degradation ladder (when a probe fails or the toolchain is absent):
1. same claim via a core-available pattern (cross-multiply; `omega`; `simp`);
2. `sorry` + gap naming the missing development
   (`-- GAP: requires field_simp (mathlib)`);
3. Tier 2 reported as "not run; project ready for `lake build`".

Never silently drop a lemma, and never report an uncompiled `decide` proof as settled: a claim is *proved* only after `lake build` succeeds. (Run D's kernel refuted a cross-multiplied lemma whose Python-side value was correct —
the translation itself was the error; only compilation catches that class.)

Translation:
- Closed computation: `theorem power_cross : 12 * 12 = 36 * 4 := by decide`
  (reading named in or after the lemma)
- Symbolic/unproven: `theorem power_symbolic : ∀ V R : Rat, R ≠ 0 → V^2/R = V*(V/R) := by sorry  -- GAP: requires field_simp/ring (mathlib) or manual calc`
- Empirical premise: `axiom ohms_law_empirical : Prop  -- assumed: V=I·R from physics, not derived`
- Definition: `def power (V R : Rat) : Rat := V^2 / R`

Spec JSON for `scaffold_lean_project.py`: `{"module":"TCT","imports":[],
"definitions":[{"name":"...","type":"...","value":"..."}],
"axioms":[{"name":"...","type":"...","note":"..."}],
"lemmas":[{"name":"...","type":"...","proof":"decide" or null,"gap":"... or null"}]}`.

9. Worked examples

A. Chained algebra (Ohm's law). Given V=12 V, R=4 Ω, power P? Steps: I = V/R = 12/4 = 3 A; P = V·I = 36 W; check P = V²/R = 144/4 = 36 W. Column 3:

`from fractions import Fraction; V=Fraction(12); R=Fraction(4); I=V/R; P=V*I; assert P==V*V/R; print(f"I={I}\nP={P}")`. Verify: `verify_column3.py --key P --expect 36` → VERIFIED.

B. Lean tier (same example). `current_value` proves with `decide` on the cleared form (`12 = 3 * 4`); `power_value` likewise via cross-multiplication (`12 * 12 = 36 * 4`, witnessing P = V²/R = 36 W); `power_symbolic` (∀ V R, R≠0 → V²/R = V·(V/R)) is `sorry` with gap "requires field_simp/ring (mathlib) or manual calc"; `ohms_law_empirical` is an axiom. Scaffold → `lake build` succeeds; 1 sorry remains = development required.

C. Refusal. Given V=12, I=5, R=4: `REFUSED: INCONSISTENT_GIVENS — Ohm's law
requires V=I·R=20 V, but V is given as 12 V; no single derivation satisfies both.`

10. Bundled scripts

If a `scripts/` directory is not present next to this file, materialize the five
scripts from §13 (write each code block to `scripts/<filename>.py`) before use —

this document is self-contained.

- `scripts/verify_column3.py` — isolated Python re-derivation checker.
  `python3 verify_column3.py SCRIPT.py --key P --expect 36` (or `--code "..."`,
  `--timeout N`). Exit 0/1/2 = VERIFIED/MISMATCH/ERROR.
- `scripts/scaffold_lean_project.py` — JSON spec → Lake project +
  `SORRY_LEDGER.md`. `--spec spec.json --out tct_lean/ [--with-mathlib]
  [--toolchain leanprover/lean4:vX.Y.Z]`. Pure core by default; lakefile
  targets current Lake (verified on Lake 5.0.0 / Lean 4.35.0).
- `scripts/lean_ledger.py` — scan a project for `sorry`/`axiom`; `--build` runs
  `lake build` if present.
- `scripts/cent_cost.py` — count ¢ primitives in a decomposition and diff
  before/after an optimization iteration (`--before v1.json --after v2.json`).
- `scripts/lean_probe.py` — run-time environment probe: checks tactic /
  notation availability on the local toolchain against the §8 baseline and
  prints the live landscape. Run before writing lemmas (probe-first).

11. The optimization loop (post-VERIFIED iteration)

Once an answer is VERIFIED (Tier 1 and, if present, Tier 2), enter a bounded
optimization loop — GLM's iteration discipline:

1. **Measure cost AND verification strength.** Record (a) the step/primitive
   count (`scripts/cent_cost.py`) and (b) the number of independent verification
   paths — cross-check `assert`s and alternative derivation paths (e.g. energy
   path vs. Newton path vs. Riemann bracket). Both are baselines.
2. **Propose a simplification.** Remove a redundant or stitched step, merge two

adjacent primitives, or re-express the derivation with fewer operations. **A simplification must not remove an independent verification path or cross-check assert** — merging redundant computation is allowed; dropping a witness path is rejected even if the final value still matches.

3. **Re-verify from scratch.** Re-run Column 3 in isolation and rebuild the Lean project. A simplification is accepted **only** if it remains VERIFIED, produces the identical final value, AND the verification-path count is preserved or increased.
4. **Record or reject.** If accepted, record the new (lower) cost and repeat. If rejected (regression, loss of verification, or dropped cross-check), revert to the prior version.
5. **Stop.** When an iteration finds no reduction, or a declared iteration bound is reached, stop. Report: original N steps → final M steps, cross-checks preserved (X→X), iterations run, final VERIFIED. The loop never changes the answer — only the proof length.

12. cent_compiler cross-domain mode (£ → ¢ → Δ)

Use when a problem is hard in its stated domain and a cross-domain analogy may make it tractable (GLM cent_compiler_v12, study GLM-STUDY-CENT-COMPILER-001).

- **£ (Abstract Intent).** State the user's problem as an operation/goal to compile.
- **¢ (Intermediate Representation).** Decompose £ into the 22-primitive vocabulary — 16 atomic (`sum, neg, mult, scale, accum, rate, wrap, spatial, exp, log, sat, hysteresis, grad, div_spatial, curl, delay`) + 6 composite (`recip, div, sqrt, eq, translinear, state_space`). Each primitive has an exact rational definition; irrational values are carried by rational pulse-density convergents (delta-sigma relaxation, no floats — UBP D1).
- **Δ (Physical Substrates).** Realize each primitive isomorphically across domains: electrical, mechanical, fluidic, thermal, optical, digital,

  spintronic, biochemical. A problem hard in one domain may be trivial in
  another — solve in the easier domain, then map the solution back via the
  isomorphism. The Knowledge Fabric (31 wheels W1–W35, 72 physical axioms)
  licenses each analogy; an unlicensed analogy is recorded as `sorry`/gap, not
  asserted.
- **Output.** Show the ¢ decomposition, at least two Δ domain isomorphisms, t
he
  solution in the chosen domain, the inverse mapping back, and a ledger:
  licensed analogies (axiom) vs. conjectural (sorry + gap). Then run the
  optimization loop (§11) over the primitive count.

13. Bundled scripts — self-contained source (materialize if absent)

If `scripts/` does not exist next to this file, create it and write each block
below to `scripts/<filename>.py` (stdlib only, no installation needed). These
are the exact sources; do not re-derive them.

scripts/verify_column3.py

```python
#!/usr/bin/env python3
"""Verify a Column-3 re-derivation script by running it in an isolated subprocess.
Usage: python3 verify_column3.py SCRIPT.py [--expect EXPECTED] [--timeout
N] [--key K]
       python3 verify_column3.py --code "print(2+2)" [--expect 4]
Exit: 0=VERIFIED, 1=MISMATCH, 2=error/timeout.
"""
import argparse, subprocess, sys, tempfile, os

def last_meaningful_line(text):
    for line in reversed(text.splitlines()):
        s = line.strip()
        if s and not s.startswith("#"):
            return s
    return ""
```

```
def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("script", nargs="?")
    ap.add_argument("--code")
    ap.add_argument("--expect")
    ap.add_argument("--timeout", type=int, default=30)
    ap.add_argument("--key")
    args = ap.parse_args()
    if not args.script and not args.code:
        ap.error("provide a script path or --code")
    if args.code:
        tmp = tempfile.NamedTemporaryFile("w", suffix=".py", delete=False)
        tmp.write(args.code); tmp.close()
        path, cleanup = tmp.name, True
    else:
        path, cleanup = os.path.abspath(args.script), False
    try:
        cwd = os.path.dirname(path) if not args.code else tempfile.gettempdir()
        proc = subprocess.run([sys.executable, path], capture_output=True,
                              text=True, timeout=args.timeout, cwd=cwd)
    except subprocess.TimeoutExpired:
        print(f"ERROR: timed out after {args.timeout}s"); return 2
    except OSError as e:
        print(f"ERROR: could not run: {e}"); return 2
    finally:
        if cleanup:
            try: os.unlink(path)
            except OSError: pass
    if proc.returncode != 0:
        print(f"ERROR: exit code {proc.returncode}")
        if proc.stderr: print(proc.stderr.strip())
        return 2
    out = proc.stdout.strip()
    if args.key:
        actual = None
        for line in out.splitlines():
```

```
            if "=" in line:
                k, _, v = line.partition("=")
                if k.strip() == args.key:
                    actual = v.strip(); break
        if actual is None:
            print(f"ERROR: no key '{args.key}'"); print(out); return 2
    else:
        actual = last_meaningful_line(out)
    if args.expect is None:
        print("OUTPUT:"); print(out); print(f"FINAL: {actual!r}"); return 0
    expected = args.expect.strip()
    if actual == expected:
        print(f"VERIFIED: {actual!r} == {expected!r}"); return 0
    print(f"MISMATCH: {actual!r} != {expected!r}"); print(out); return 1

if __name__ == "__main__":
    sys.exit(main())
```

scripts/scaffold_lean_project.py

```python
#!/usr/bin/env python3
"""Scaffold a Lean 4 Lake project from a TCT lemma spec.
Spec JSON: {"module":"TCT","imports":[],"definitions":[{"name","type","value"}],
"axioms":[{"name","type","note"}],"lemmas":[{"name","type","proof" or null,"gap" or
 null}]}
proof=null -> sorry + GAP comment. Writes lean-toolchain=leanprover/lean4:sta
ble (elan resolves it).
Lakefile targets current Lake: single lean_lib with an explicit glob, no lean_exe
(verified Lake 5.0.0 / Lean 4.35.0; the pre-5.0 lean_exe template fails package c
onfig).
Usage: python3 scaffold_lean_project.py --spec spec.json --out proj/ [--with-ma
thlib] [--toolchain TAG]
"""
import argparse, json, os, sys
```

```
LAKEFILE = """import Lake
open Lake DSL
package {name} where
  moreServerArgs := #["-DwarningAsError=false"]
@[default_target]
lean_lib {mod} where
  globs := #[.one `{mod}.Basic]
"""
TOOLCHAIN_DEFAULT = "leanprover/lean4:stable"

def render_lemma(lem):
    name, typ = lem["name"], lem["type"].strip()
    proof, gap = lem.get("proof"), lem.get("gap")
    if proof and proof.strip():
        return f"theorem {name} : {typ} := by\n  {proof.strip()}\n"
    gap_line = f"  -- GAP: {gap}" if gap else "  -- GAP: (no description)"
    return f"theorem {name} : {typ} := by\n  sorry\n{gap_line}\n"

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--spec", required=True)
    ap.add_argument("--out", required=True)
    ap.add_argument("--name", default="tct")
    ap.add_argument("--toolchain", default=TOOLCHAIN_DEFAULT)
    ap.add_argument("--with-mathlib", action="store_true")
    args = ap.parse_args()
    spec = json.load(open(args.spec))
    mod = spec.get("module", "TCT")
    imports = list(spec.get("imports", []))
    if args.with_mathlib:
        imports = ["Mathlib"] + [i for i in imports if i != "Mathlib"]
    os.makedirs(os.path.join(args.out, mod), exist_ok=True)
    open(os.path.join(args.out, "lean-toolchain"), "w").write(args.toolchain + "\n")
    lakefile = LAKEFILE.format(name=args.name, mod=mod)
    if args.with_mathlib:
```

```
        lakefile = lakefile.replace("package " + args.name + " where",
            f'require mathlib from git "https://github.com/leanprover-community/mathlib4" @ "{args.toolchain.split(":")[-1]}"\n\npackage {args.name} where')
    open(os.path.join(args.out, "lakefile.lean"), "w").write(lakefile)
    lines = []
    for imp in imports: lines.append(f"import {imp}")
    if imports: lines.append("")
    lines.append(f"namespace {mod}\n")
    proved = sorry = axioms = defs = 0
    sorry_list, axiom_list = [], []
    for d in spec.get("definitions", []):
        defs += 1; lines.append(f"def {d['name']} : {d['type']} := {d['value']}")
    if defs: lines.append("")
    for a in spec.get("axioms", []):
        axioms += 1; lines.append(f"axiom {a['name']} : {a['type']}")
        if a.get("note"): lines.append(f"-- assumed premise: {a['note']}")
        axiom_list.append((a["name"], a["type"], a.get("note", "")))
    if axioms: lines.append("")
    for lem in spec.get("lemmas", []):
        if lem.get("proof"): proved += 1
        else:
            sorry += 1; sorry_list.append((lem["name"], lem["type"], lem.get("gap", "")))

        lines.append(render_lemma(lem))
    lines.append(f"end {mod}\n")
    open(os.path.join(args.out, mod, "Basic.lean"), "w").write("\n".join(lines))
    ledger = [f"# Development-required ledger — {args.name}", "",
        f"- lemmas total: {proved+sorry}", f"- proved (sorry-free): **{proved}**",
        f"- `sorry` (development required): **{sorry}**", f"- axioms: **{axioms}**", f"- definitions: {defs}", ""]
    if sorry_list:
        ledger += ["## sorry lemmas", ""]
        for n, t, g in sorry_list: ledger += [f"- `{n}` : `{t}`", f"  - gap: {g or '(undescribed)'}"]
        ledger.append("")
    if axiom_list:
```

```
        ledger += ["## axioms (assumed)", ""]
        for n, t, note in axiom_list: ledger.append(f"- `{n}` : `{t}` — {note}")
        ledger.append("")
    fence = chr(96) * 3
    ledger += ["## To verify", "", fence + "bash", f"cd {args.out} && lake build", fen
ce,
        "0 sorry = fully described; each sorry above is exactly what remains."]
    open(os.path.join(args.out, "SORRY_LEDGER.md"), "w").write("\n".join(ledger) +
"\n")
    print(f"Scaffolded {args.out}: {proved} proved, {sorry} sorry, {axioms} axioms, {
defs} defs; toolchain {args.toolchain}")
    return 0

if __name__ == "__main__":
    sys.exit(main())
```

scripts/lean_ledger.py

```python
#!/usr/bin/env python3
"""Scan a Lean project for sorry/admit/axiom; optionally run `lake build`.
Usage: python3 lean_ledger.py PROJECT_DIR [--build]
"""
import argparse, os, re, subprocess, sys

SORRY_LINE = re.compile(r"^\s*(sorry|admit)\s*$")  # proof-body line only; ignore
s comments/identifiers
AXIOM_RE = re.compile(r"^\s*axiom\s+(\w+)")
DECL_RE = re.compile(r"^\s*(theorem|lemma|def|example|axiom)\s+(\w+)")

def scan_file(path):
    sorry, axioms, current = [], [], None
    for line in open(path):
        m = DECL_RE.match(line)
        if m: current = m.group(2)
```

```
        if AXIOM_RE.match(line): axioms.append(AXIOM_RE.match(line).group(1))
        if SORRY_LINE.match(line) and current: sorry.append(current)
    return sorry, axioms

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("project")
    ap.add_argument("--build", action="store_true")
    args = ap.parse_args()
    all_sorry, all_axioms, n = [], [], 0
    for root, _, files in os.walk(args.project):
        for fn in files:
            if fn.endswith(".lean"):
                n += 1
                s, a = scan_file(os.path.join(root, fn))
                all_sorry += s; all_axioms += a
    print(f"Scanned {n} file(s): sorry={len(all_sorry)} {sorted(set(all_sorry)) or ''}, axio
ms={len(all_axioms)} {sorted(set(all_axioms)) or ''}")
    if args.build:
        on_path = any(os.path.exists(os.path.join(p, "lake")) for p in os.environ.get("P
ATH", "").split(os.pathsep))
        if not on_path:
            print("BUILD: lake not on PATH — project ready for `lake build` elsewhere.
"); return 0
        proc = subprocess.run(["lake", "build"], cwd=args.project, capture_output=Tr
ue, text=True, timeout=600)
        print(proc.stdout)
        if proc.returncode == 0: print("BUILD: succeeded."); return 0
        print("BUILD FAILED:"); print(proc.stderr); return 1
    return 0

if __name__ == "__main__":
    sys.exit(main())
```

scripts/cent_cost.py

```python
#!/usr/bin/env python3
"""cent_compiler cost metric + optimization-loop diff.
Vocabulary (22): atomic=sum neg mult scale accum rate wrap spatial exp log sa
t
hysteresis grad div_spatial curl delay; composite=recip div sqrt eq translinear sta
te_space.
Usage: python3 cent_cost.py decomp.json
       python3 cent_cost.py --before v1.json --after v2.json
JSON: {"primitives": ["sum","mult",...]}
"""
import argparse, json, sys

ATOMIC = {"sum","neg","mult","scale","accum","rate","wrap","spatial","exp","log",
          "sat","hysteresis","grad","div_spatial","curl","delay"}
COMPOSITE = {"recip","div","sqrt","eq","translinear","state_space"}
VOCAB = ATOMIC | COMPOSITE

def count(path):
    spec = json.load(open(path))
    prims = spec.get("primitives", [])
    unknown = [p for p in prims if p not in VOCAB]
    a = sum(1 for p in prims if p in ATOMIC)
    c = sum(1 for p in prims if p in COMPOSITE)
    return prims, a, c, unknown

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("decomp", nargs="?")
    ap.add_argument("--before"); ap.add_argument("--after")
    args = ap.parse_args()
    if args.before and args.after:
        pb, ab, cb, ub = count(args.before)
        pa, aa, ca, ua = count(args.after)
        print(f"BEFORE: total={len(pb)} (atomic={ab}, composite={cb})" + (f" WARNI
```

```
NG unknown: {ub}" if ub else ""))
        print(f"AFTER:  total={len(pa)} (atomic={aa}, composite={ca})" + (f" WARNIN
G unknown: {ua}" if ua else ""))
        delta = len(pb) - len(pa)
        if delta > 0: print(f"REDUCED by {delta} — re-verify (Column 3 + Lean) befo
re accepting.")
        elif delta == 0: print("NO REDUCTION — loop may stop.")
        else: print(f"REGRESSED by {-delta} — reject.")
        return 0 if delta >= 0 else 1
    if not args.decomp: ap.error("provide a decomp file or --before/--after")
    prims, a, c, unknown = count(args.decomp)
    print(f"primitives: total={len(prims)} (atomic={a}, composite={c})")
    if unknown: print(f"WARNING unknown: {unknown}"); return 1
    return 0

if __name__ == "__main__":
    sys.exit(main())
```

scripts/lean_probe.py

```python
#!/usr/bin/env python3
"""Probe the local Lean toolchain for the TCT lemma patterns (probe-first, S8).
Writes tiny probe files to a temp dir, runs `lean` on each, and prints the live
landscape vs. the pure-core 4.35.0 baseline. Believe the probe, not the docs.
Usage: python3 lean_probe.py [--lean /path/to/lean]
Exit: 0 probe ran, 3 no Lean found.
"""
import argparse, os, shutil, subprocess, sys, tempfile

PROBES = [
    # (name, source, baseline on pure-core Lean 4.35.0: True = compiles)
    ("decide_nat", "example : 12 * 12 / 4 = 36 := by decide", True),
    ("rfl_definitional", "example : 1 + 1 = 2 := by rfl", True),
    ("omega_int_mod", "example : ∀ n : Int, (n + 1) % 7 = (n % 7 + 1) % 7 := by ome
```

```
ga", True),
    ("simp", "example : 2 + 2 = 4 := by simp", True),
    ("Q_notation", "example : (12 : ℚ)^2 / 4 = 36 := by decide", False),
    ("rat_decide", "example : (12 : Rat)^2 / 4 = 36 := by decide", False),
    ("rat_rfl", "example : (12 : Rat)^2 / 4 = 36 := by rfl", False),
    ("rat_native_decide", "example : (12 : Rat)^2 / 4 = 36 := by native_decide", True)
,
    ("norm_num", "example : 2 + 2 = 4 := by norm_num", False),
    ("ring", "example (a b : Nat) : a + b = b + a := by ring", False),
    ("nat_factorial", "example : Nat.factorial 5 = 120 := by rfl", False),
    ("real_pi", "#check Real.pi", False),
    ("nat_xor_variable", "example (a : Nat) : (a ^^^ 0) = a := by decide", False),
]

def find_lean(explicit):
    if explicit:
        return explicit
    on_path = shutil.which("lean")
    if on_path:
        return on_path
    elan = os.path.join(os.environ.get("ELAN_HOME", ""), "bin", "lean")
    return elan if os.path.exists(elan) else None

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--lean")
    args = ap.parse_args()
    lean = find_lean(args.lean)
    if not lean:
        print("NO LEAN on PATH or ELAN_HOME - Tier 2 unavailable here.")
        print("Degrade per S8: sorry+gap for formal steps; report 'lake build: not run'
.")
        return 3
    ver = subprocess.run([lean, "--version"], capture_output=True, text=True)
    print("probing: " + (ver.stdout.strip() or ver.stderr.strip()))
    tmp = tempfile.mkdtemp(prefix="tct_probe_")
```

```
    try:
        agree = disagree = 0
        for name, src, baseline in PROBES:
            path = os.path.join(tmp, name + ".lean")
            with open(path, "w", encoding="utf-8") as f:
                f.write(src + "\n")
            r = subprocess.run([lean, path], capture_output=True, text=True, timeout
=120)
            live = (r.returncode == 0)
            mark = "AGREE" if live == baseline else "!= BASELINE"
            if live == baseline:
                agree += 1
            else:
                disagree += 1
            print("  %-20s %-9s (baseline %-3s) %s" % (name, "AVAILABLE" if live els
e "absent", "yes" if baseline else "no", mark))
        print("%d agree with the 4.35.0 baseline, %d differ - trust the probe." % (ag
ree, disagree))
        print("Kernel-checked rationals: cross-multiply into Nat (S8); native_decide i
s compiler-trusted.")
        return 0
    finally:
        shutil.rmtree(tmp, ignore_errors=True)

if __name__ == "__main__":
    sys.exit(main())
```

14. Optimal construction — the four conditions (runbook)

Derived by applying TCT to itself (a self-referential run on "how should TCT be run"), verified in an isolated subprocess. An optimal run satisfies four exact, checkable conditions — these are what produce interesting insights rather than

just verified answers.

1. **Classification identity.** Every atomic claim lands in exactly one class: proved / sorry / axiom / refused, so `n = p + s + a + r`. Decompose and classify *before* writing any column. For "why/how" questions, declare illustrative givens explicitly and engineer them as rational witnesses (UBP drives this: choose values so every quantity stays in ℚ; carry v^2 instead of v when a root is unavoidable).
2. **Independent cross-checks.** If `c` cross-check paths are written and `t` tautologies are removed, insight of the cross-check class requires `(c − t) ⩾ 2`. Audit checks for tautology as a final step: a real check compares two expressions with disjoint derivation paths; anything algebraicall
y
   equivalent by construction is a tautology — remove it, don't count it.
3. **Optimization-loop acceptance.** A simplification is accepted iff cost `k` (¢ primitives or steps) decreases **and** the verification-path count is preserved or increased. A reduction that drops a witness is rejected even if the final value still matches. Run one iteration, then stop — the loop never changes the answer, only proof length.
4. **Named ledger.** Every sorry names the exact missing tactic or lemma; ever
y
   axiom is named; an unnamed assumed premise is `WOULD_BE_GUESS`. `0 so
rry =
   fully described`. The ledger includes a **coverage map** (§6): which claims carry a Lean lemma (k of N); Python-only steps are listed with the reason. When stuck, switch domains via cent mode (⩾2 domains; license each analogy as axiom or mark it conjectural sorry).

Report the four ledgers (proved / sorry / axiom / refused) plus a short insights paragraph. The meta-run also fixed a scanner defect: `lean_ledger.py` now counts only standalone `sorry`/`admit` proof-body lines, so comments and identifiers containing the substring "sorry" no longer false-positive.